Task 3: Functional Components in React

Summary

React applications are built using components, which are the fundamental building blocks of the user interface (UI). A component is a reusable piece of code that represents a specific part of the application, such as a navigation bar, profile card, button, or footer. By dividing the UI into smaller components, React makes applications easier to develop, maintain, and reuse.

A React component is simply a JavaScript function that returns JSX. Component names must always begin with a capital letter, otherwise React treats them as normal HTML elements. The main component of a file is generally exported using the export default keyword so that it can be imported and used in other files.

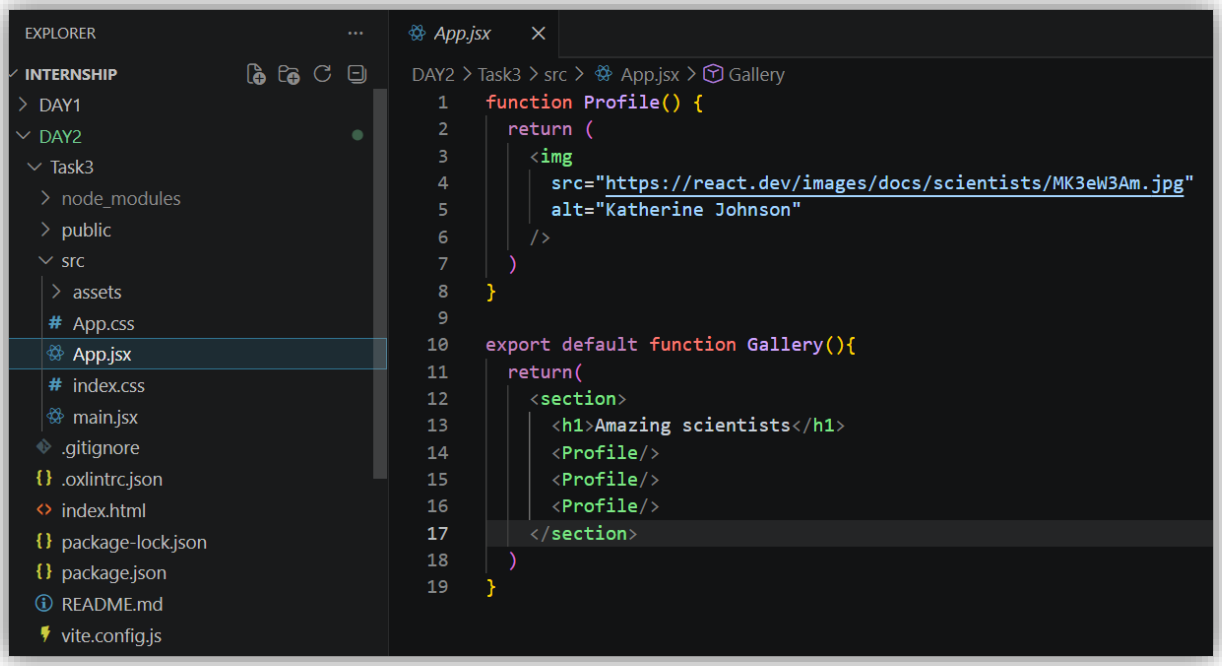
Components can be used inside other components, allowing developers to create complex user interfaces by combining multiple smaller components. This relationship is known as parent and child components. A parent component can render one or more child components, and the same child component can be reused multiple times within the application.

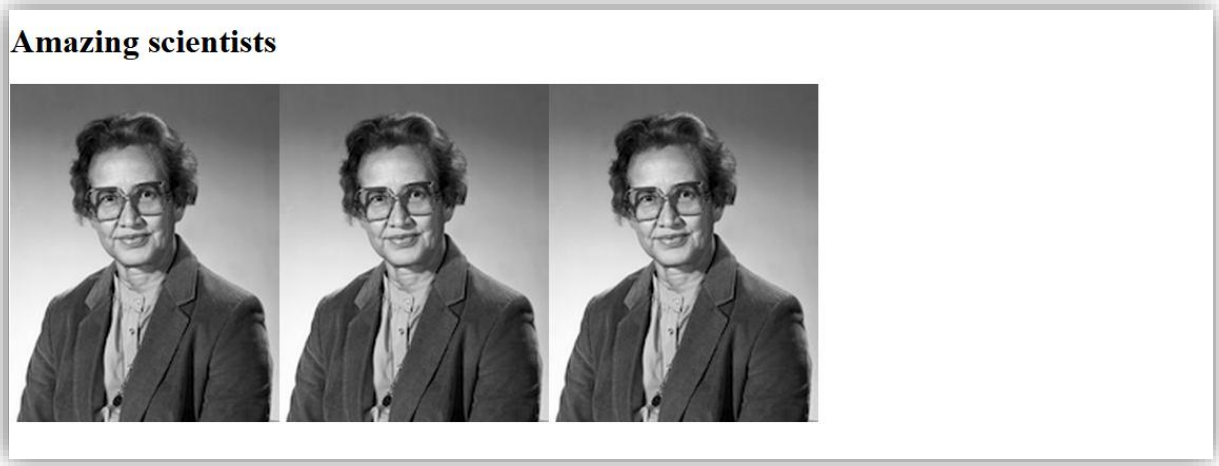
When writing components, the returned JSX should be enclosed within parentheses if it spans multiple lines. Also, components should always be declared at the top level instead of defining one component inside another, as nested component definitions can lead to performance issues and unexpected behavior.

React follows the principle of "Write Once, Reuse Anywhere." Since components are reusable, they help reduce code duplication, improve readability, and make applications easier to manage as they grow.

Learning Outcome

After completing this task, I learned the concept of React Functional Components, how to create and export components, the importance of capitalized component names, the parent-child component relationship, component reusability, and the best practices for organizing React components.


EXPLORER
INTERNSHIP
DAY1
DAY2
Task3
node_modules
public
src
assets
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
App.jsx
DAY2 > Task3 > src > App.jsx > Gallery
function Profile() {
return (
<img
src="https://react.dev/images/docs/scientists/MK3eW3Am.jpg"
alt="Katherine Johnson"
/>
)
}
export default function Gallery(){
return(
<section>
<h1>Amazing scientists</h1>
<Profile/>
<Profile/>
<Profile/>
</section>
)
}


Amazing scientists

Task 4: Passing Props to a Component

Summary

Props (Properties) are used in React to pass data from a parent component to a child component. They allow components to communicate with each other and make components reusable by displaying different data without changing the component's internal code.

Props work similarly to function arguments. A parent component passes values such as strings, numbers, objects, arrays, functions, or even JSX to a child component through attributes. The child component receives these values as a single props object or by using destructuring, making the code cleaner and easier to read.

React also allows developers to specify default values for props. If a parent component does not provide a particular prop, the default value is automatically used. This helps prevent errors and ensures that components behave consistently.

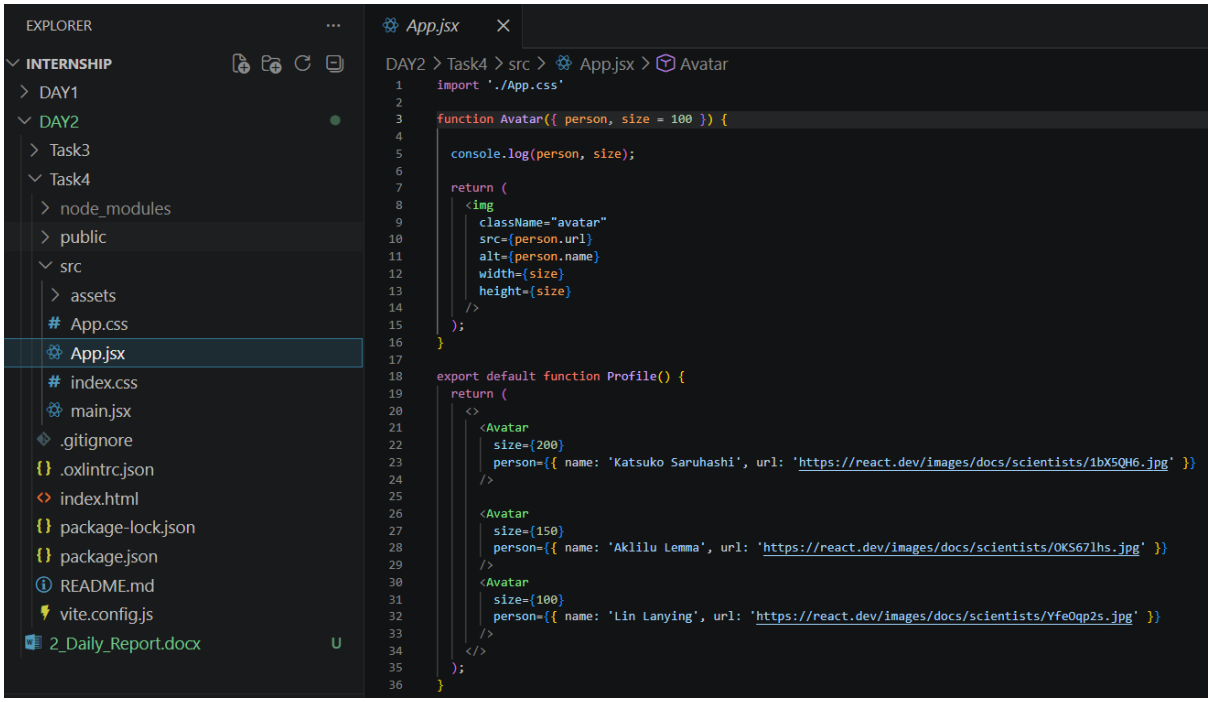
Another useful feature is the children prop, which allows a component to wrap and display any nested JSX passed between its opening and closing tags. This makes wrapper components such as cards, layouts, and containers more flexible and reusable.

React also supports the spread operator (...props), which forwards all props from one component to another without passing each prop individually. This feature should be used carefully to keep the code readable and maintainable.

An important characteristic of props is that they are read-only (immutable). A child component should never modify the props it receives. Whenever the parent component passes new values, the child component automatically receives the updated props and re-renders accordingly.

Learning Outcome

After completing this task, I learned how to pass data between components using props, read props through destructuring, use default prop values, pass JSX using the children prop, forward props with the spread operator, and understand that props are immutable and controlled by the parent component.



⇨ **GITHUB REPO LINK**
https://github.com/yashrathod910/15D_Internship/tree/main/DAY2